

How every node in Spice Restaurent AI works

A plain-language tour of your WhatsApp chatbot — what each of the 13 nodes does, which settings we chose and why, what we deliberately left off and why, and how the Meta WhatsApp Cloud API is wired to it.

13 nodes

Gemini 2.5-flash

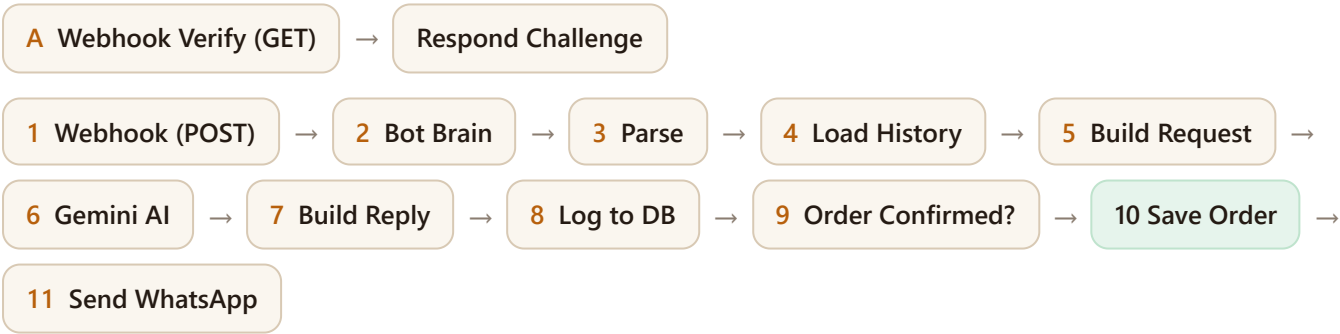
PostgreSQL memory

Bilingual · English + Urdu

Status: Live

The two paths through the workflow

Every WhatsApp event hits n8n. A GET is Meta checking you're alive; a POST is a real message.



The verification path

This runs once, when you paste the webhook URL into Meta. It never runs again for real messages.

A Webhook Verify (GET)

WEBHOOK NODE · GET

When you save the callback URL in Meta, Meta sends a one-time **GET** request to prove the address is really yours. This node catches that check.

◆ WHAT WE SET & WHY

- **Method = GET, Path = `whatsapp`** — gives the fixed URL `/webhook/whatsapp` that you paste into Meta.
- **Respond = "Using Respond to Webhook node"** — we need to hand back Meta's exact challenge value, so we let the next node build the reply instead of a canned response.

◇ WHAT WE'RE NOT DOING & WHY

- No authentication on the node. *Meta's own verify-token (`resto_verify_123`) is the security check — Meta must echo it back, so an extra password would only get in the way.*

A Respond Challenge

RESPOND TO WEBHOOK NODE

Meta's verification GET carries a value called `hub.challenge`. To pass the check, we must reply with that *exact* value as plain text. This node does only that.

◆ WHAT WE SET & WHY

- **Respond With = Text**, body = `{{ $json.query['hub.challenge'] }}` — echoes Meta's challenge straight back so Meta marks the webhook **Verified**.

◇ WHAT WE'RE NOT DOING & WHY

- Not returning JSON or a status page. *Meta wants the raw challenge string and nothing else — extra wrapping makes verification fail.*

The message path

This is the real chatbot. It runs top-to-bottom every time a customer sends a WhatsApp message.

1 Webhook Messages (POST)

WEBHOOK NODE · POST

The front door. Every incoming customer message arrives here as a **POST** from Meta.

◆ WHAT WE SET & WHY

- **Method = POST**, same **Path = whatsapp** — one URL handles both Meta's verify (GET) and real messages (POST).
- **Respond = Immediately, code 200** — we tell Meta "got it" the instant the message lands, then keep working in the background. *If we made Meta wait for Gemini, Meta would think we timed out and resend the same message over and over.*

◇ WHAT WE'RE NOT DOING & WHY

- Not verifying Meta's signature header. *Fine for this private test setup; worth adding if you go fully public with a real business number.*

2 Bot Brain

SET NODE · THE EDITABLE PROMPT

This is the one node **you** edit. It holds the entire personality, menu, prices, rules and language instructions as a single block of text called `systemPrompt`. Everything the bot "knows" lives here.

◆ WHAT WE SET & WHY

- **One field, `systemPrompt`** — kept as plain text so you can change the menu or tone with no code.
- **"Include other input fields" = ON** — passes the incoming message through untouched alongside the prompt, so nothing downstream breaks.

◇ WHAT WE'RE NOT DOING & WHY

- Prices/menu aren't in the code or database. *On purpose* — one text box means you never need a developer to change a price.

 To change the menu: open n8n → this node → edit the text → Save. Live instantly.

3 Parse Message

CODE NODE · JAVASCRIPT

Meta's message packet is large and messy. This node digs out just what we need: **who** sent it, their **name**, and the **text** they typed.

◆ WHAT WE SET & WHY

- Pulls **sender number, profile name, message text, message ID** into clean fields.
- Handles **button/list taps** too (uses the button's title as the text).
- **Ignores non-messages** — Meta also sends "delivered / read" receipts. If there's no real message, the node stops the run early (`return []`) so the bot doesn't reply to a receipt.

◇ WHAT WE'RE NOT DOING & WHY

- Not downloading images/voice notes. *It notes their type (e.g. "[image message received]") but doesn't process the file — text is all this bot needs today.*

4 Load History

POSTGRES NODE · SELECT

This is the bot's **memory**. It reads the last few messages you exchanged with *this* customer from the database, so the bot remembers the conversation instead of treating every message as brand new.

◆ WHAT WE SET & WHY

- **Query the last 8 exchanges** for this exact phone number, newest first.
- **"Always output data" = ON** — a brand-new customer has no history; this keeps the flow moving instead of stopping on an empty result.

◇ WHAT WE'RE NOT DOING & WHY

- Not loading the *whole* chat history. *Only 8 recent turns — enough for context, while keeping the AI fast and within limits.*

5 Build Request

CODE NODE · JAVASCRIPT

Assembles the package we send to the AI. It takes the remembered history plus the new message and stacks them in order — **customer, bot, customer, bot...** — so Gemini reads the conversation like a real transcript.

◆ WHAT WE SET & WHY

- Flips the history to **oldest-first** and tags each line as **customer** or **bot**, then adds the new message last.
- Carries the **Bot Brain prompt** along so the AI keeps its rules.

◇ WHAT WE'RE NOT DOING & WHY

- No summarising or trimming of old messages. *Not needed at 8 turns; would only add complexity.*

6 Gemini AI

HTTP REQUEST · GOOGLE GEMINI

The brain in action. Sends the prompt + conversation to Google's **gemini-2.5-flash** model and gets the written reply back.

◆ WHAT WE SET & WHY

- **Model 2.5-flash** — fast and free-tier friendly; ideal for chat.
- **Temperature 0.6** — friendly and natural, but still sticks to your prices and rules (not too random).
- **Max 800 tokens** — long enough for an order summary, short enough to stay snappy.
- **Auto-retry up to 5×** — if Google's free tier is briefly busy, it waits and tries again instead of going silent.

◇ WHAT WE'RE NOT DOING & WHY

- Not on a paid plan. *Free tier allows ~20 replies/minute — plenty for a real restaurant. Turn on billing later if you expect bursts.*

7 Build Reply

CODE NODE · JAVASCRIPT

Cleans up the AI's answer and does one clever thing: it looks for a **hidden order line**. When a customer confirms, the AI secretly adds a machine line starting with `#ORDER#`. This node pulls that out, turns it into structured order data, and *removes* it so the customer never sees it.

◆ WHAT WE SET & WHY

- Extracts the `#ORDER#` data (items, total, type, address, name) into a clean order object.
- **Strips that hidden line** from the visible reply.
- Provides a polite **fallback message** if the AI ever returns nothing.

◇ WHAT WE'RE NOT DOING & WHY

- Not trusting the AI to write to the database directly. *We parse and validate the order here first — safer and predictable.*

8 Log to DB

POSTGRES NODE · INSERT

Saves every exchange (customer message + bot reply) into the `chat_logs` table. This is what makes memory possible next time — and gives you a full record of every conversation.

◆ WHAT WE SET & WHY

- Inserts **sender, name, their text, the reply, message ID** into `chat_logs`.
- **"Continue on error" = ON** — if a save hiccups, the customer *still* gets their reply. The message matters more than the log.

◇ WHAT WE'RE NOT DOING & WHY

- Not deleting old logs. *Kept for history & memory; you can clear them anytime with one SQL command.*

9 Order Confirmed?

IF NODE · THE FORK

The decision point. It checks: did Build Reply find a real order? **Yes** → go save it, then reply. **No** → skip straight to replying.

◆ WHAT WE SET & WHY

- One boolean test: **"is there an order?"** — true sends the flow to **Save Order**, false goes straight to **Send WhatsApp**.

◇ WHAT WE'RE NOT DOING & WHY

- Not saving every chat as an order. *Only genuine confirmations (customer said YES) reach the orders table — no junk rows.*

10 Save Order

POSTGRES NODE · INSERT (ORDERS)

Writes a confirmed order into the **orders** table — a proper record you can act on in the kitchen.

◆ WHAT WE SET & WHY

- Saves **customer, phone, items, total (PKR), order type, address**, and marks status **"confirmed"**.
- Items stored as structured **JSON** so you can read them back cleanly.

◇ WHAT WE'RE NOT DOING & WHY

- Not charging payment or printing a receipt. *This records the order; connecting a POS or printer is a future upgrade.*

11 Send WhatsApp

HTTP REQUEST · META CLOUD API

The last step — sends the finished reply back to the customer on WhatsApp through Meta's Cloud API.

◆ WHAT WE SET & WHY

- **POST to Meta's Graph API** with your **permanent token** in the Authorization header.
- Sends a plain **text** message to the customer's number with the cleaned reply.
- **"Continue on error" = ON** — one failed send never crashes the whole workflow.

◇ WHAT WE'RE NOT DOING & WHY

- Not sending buttons/images/templates yet. *Plain text is simplest and works inside the 24-hour reply window; rich formats can be added later.*

How the Meta WhatsApp Cloud API is wired up

These are the one-time setup steps that connect Meta's servers to your n8n workflow. Once done, they keep working on their own.

1 A Meta app with WhatsApp added

In developers.facebook.com you created an app and added the **WhatsApp** product. This gives you a free **test phone number** and the two IDs the bot uses: the **Phone Number ID** (who sends) and the **WABA ID** (the business account).

2 The public tunnel (ngrok)

Meta lives on the internet; your n8n runs on your PC. **ngrok** gives your PC a permanent public web address that forwards to n8n on port 5678. Meta talks to that address; it never changes as long as you use your reserved domain.

3 Callback URL + Verify Token

In **WhatsApp** → **Configuration** you pasted the callback URL `.../webhook/whatsapp` and the verify token `resto_verify_123`. Meta sent the GET check (nodes A above), your workflow echoed the challenge, and Meta marked it **Verified**.

4 Subscribe to the "messages" field

On the same page you switched the object to **WhatsApp Business Account** and turned **messages** ON. This is what tells Meta to actually forward customer messages to your webhook — the piece that was missing at first.

5 Subscribe the app to the business account

A behind-the-scenes call linked your app to the WhatsApp Business Account (`subscribed_apps`). Without this, messages verify but never arrive. This fixed the "no reply" problem.

6 A permanent System User token

Test tokens die after 24 hours. You created a **System User** in Business Settings, gave it the WhatsApp permissions, and generated a token set to expire **Never**. That token sits in the Send WhatsApp node so the bot keeps working without daily re-logins.

7 Recipient list (test number only)

The free test number can only message up to **5 pre-verified numbers**. Each one is added under **API Setup** → **Manage phone number list** and confirmed with a code Meta sends to that WhatsApp. A real business number removes this limit.

The one thing still to do: add your three new numbers (+92 309 5595593, +92 324 1470624, +92 325 1238084) in **Meta** → **WhatsApp** → **API Setup** → **Manage phone number list**. Each needs the verification code Meta texts to that WhatsApp. This can only be done by you in Meta — it isn't something the workflow can do.